# Repaso Redes y Sistemas Distribuidos: Teórico

## CAPA DE APLICACIÓN

Enfoques para desarrollar aplicaciones de red para la internet:
1. El programador para especificar la comunicación usa una API: conj basico de funciones, como un socket API para el software en la internet.
– El saber sobre el sistema operativo de red permite al programador escribir mejor código y desarrollar aplicaciones más eficientes.
2. La Web: El programador se apoya en la tecnología de la web para construir una aplicación de red..
La Web provee servicios al software de la aplicación que facilitan la implementación de la comm y E/S asi como permitir enfocarse en el propósito de la app.
**Arquitecturas:**

| **Tópico** | **Cliente-Servidor** | **P2P** |
|---|---|---|
| **Servidor** | Siempre presente (IP fija) | Mínimo/inexistente |
| **Comm. entre clientes** | Nula | Constante |
| **Conexión entre clientes** | Intermitente | Intermitente |
| **Bajada de datos** | Pedido al servidor | Pedidos a los demás peers |
| **Tiempo de distribución** | max(N*F/Us, F/dmin) | max{F/Us, F/dmin, N*F/(Us + sumui } |

| UDP | TCP |
|---|---|
| 1. Cliente crea datagrama con IP y puerto del servidor y lo envía<br>2. Si llega el datagrama, servidor lo lee<br>3. Servidor envía respuesta especificando dirección y puerto de cliente<br>4. Si llega el datagrama, el cliente lo lee.<br>5. Cliente finaliza<br>• Evaluación:<br>– No se dice qué se hace si la respuesta no llega al cliente.<br>– Es responsabilidad de la aplicación red manejar esto. | 1. Se ejecuta proceso del servidor<br>2. Servidor espera por pedido de conexión entrante.<br>3. El cliente realiza pedido de conexión al servidor<br>4. El servidor acepta la conexión con el cliente<br>5. El cliente envía pedido al servidor<br>6. El servidor lee el pedido<br>7. El servidor envía la respuesta<br>8. El cliente lee la respuesta<br>– TCP provee transferencia de stream de bytes ordenada<br>9. Si hay más pedidos al servidor: Goto 5<br>10. El cliente cierra la conexión<br>11. El servidor cierra la conexión |

# CAPA DE TRANSPORTE

**Objetivos:** Comunicación lógica (conexión directa entre hosts) entre procesos de aplicación en sistemas finales.
**Problemas que resuelve:**

- Uso de timers y retransmisión de paqs.
- Buffering para comunicación confiable eficiente
- Control de flujo y congestión.
- Entrega ordenada segun flujo de paqs de la CA (con numeros de secuencia/timers/acks)

## TCP (Transmission Control Protocol)

Busca proporcionar un flujo de bytes confiable de extremo a extremo a través de una interfaz mala (se adapta). Entidad de transporte: ETCP.
**Problemas que resuelve:**

- Retransmisión de paquetes (con número de secuencia/acks/timers).
- Fijar la duración de temporizadores de retransmisiones
- Manejo de conexiones entre pares de procesos
- Direccionamiento
- Control de flujo y congestión

Los segmentos en TCP son de 64KiB. Y se envian estableciendo una conexión entre máquinas a través de sockets cuya dirección es IP + Puerto. Los sockets pueden usarse para varias conexiones con identificadores.
La CR no garantiza entrega apropiada => TCP usa acks para confirmar su recepción, sino retransmite; Si no hay orden correcto => TCP reensambla correctamente.
**Componentes de un segmento:**
<h> (20B (32b word) + Datos (opcional, si no los tiene son números de confirmación o msj de ctrl) + src 16b + dest 16b) + 48b (socket)

## ENTREGA DE DATOS CONFIABLE

| **Tópico** | **Parada y espera** |
|---|---|
| Cuando se usa | Baja latencia (bajo RTT, solo se manda un paq antes del ack) |
| Luego de enviar un paq | Para de enviar y espera un tiempo tazonable |
| Manejo de acks | Si llega se envía el siguiente paq, sino se retransmite |
| paq/ack demorado (no perdido) | Retransmisión duplicada (num sec) que el receptor descarta |
| rendimiento | Mejor caso (1 canal de comm, no hay errores de transmision de paqs): Uso del canal = L/R/(RTT +L/R) |

Protocolos de tuberia

| **Tópico** | **Retroceso N** | **Repetición Selectiva** |
|---|---|---|
| cuando se usa | Latencia alta (RTT > Tcopia), y se pueden mandar varios paqs antes de 1 ack | Alta latencia & muchos errores; perdidas/demoras de paqs |
| Manejo de paqs posteriores al paq malo | Se descartan todos | Se almacenan en buffer, y cuando llega el paq retransmitido, se entregan en secuencia todos |
| Tipo de confirmación | ack acumulativo (maximo numero de paqs buenos recibidos) | ack individuales/ack negativo |
| Uso de Temporizador | Para el paq mas viejo no confirmado (si expira retransmite los no confirmados).<br>si hay ack y hay no confirmados se reinicia, cc se detiene | temporizador para cada paq, y reenvia paqs sin ack/nak, |
| buffer del receptor | No necesita almacenamiento adicional | ventana corrediza p/conj. de paqs, se mantiene un paq hasta que sus anteriores fueron a la CA |
| Ventana del emisor | Tamaño N | Tamaño N |
| Ventana del receptor | solo admite 1 | (MAX_SEQ + 1)/2. |
| rendimiento | uso ineficiente de canal con segmentos perdidos/demorados | Código más complejo en el receptor para el manejo eficiente de errores,<br>a caballito para transmisión bidireccional eficiente, timer auxiliar |

# CONTROL DE FLUJO

Evitar que un host emisor rápido sature a un host receptor lento, entre 2 máquinas conectadas entre sí.

**Necesidad de control de flujo en CT:** Situaciones que el ctrl de flujo de CE no resuelve.

- CA lee msjs mucho después de su llegada (ocupada) y se satura
- CR ralentiza al receptor (router dania un paq y luego llegan otros en buen estado/alg de routing elige rutas largas y llegan paqs desordenados) almacenando paqs en buffer y disminuyendo su capacidad
- CT ralentiza al receptor con muchas conexiones abiertas (solucionable con buffer p/c/conexión mediante solicitud->reservación y ajuste dinámico de búfer)

**Control de flujo en TCP:** No se requiere inmediatez en la entrega de datos ni de recepción. Se requiere guardar datos, no segmentos, =>:
Se usa un buffer de recepción circular en el que:

- Se le indica al emisor el tamaño de ventana con un campo en el encabezado TCP de 16b. La fórmula para el tamaño de ventana es: RcvBuffer-[LasByteRcvd-LastByteRead] (si es 0 no puede enviar datos). El envío de segmentos cumple la propiedad: LastByteSent−LastByteAcked ≤ tamaño de ventana.
- El receptor coloca los datos en el buffer cuando recibió bytes en el orden correcto y en secuencia, pudiendo confirmar su llegada y anunciar un nuevo tamaño de ventana al emisor (también si está lleno o es 0).

**Pérdida de segmentos:** Se puede gestionar con:

- solicitud de segmento especial NAK (en el campo opciones si es que hay un brecha entre el número de secuencia esperado y recibido)
- Acks selectivos sobre datos específicos recibidos (habilitando Sack permitted option y Sack potion con lista de acks recibidos)

# CONTROL DE CONGESTIÓN

La congestión se da cuando el emisor envia mas información que la capacidad de carga de la subred, y no se entrega toda a la velocidad a la que llega. Entonces, TCP tiene una ventana para la congestión VC igual al tamaño de los bytes del emisor, que se ajusta cuando la congestión ocurre.

**Algoritmos para calcular VC**

1. **Arranque lento:**
   a. emisor asigna un STM a la ventana, y lo envía
   b. Si se recibe el ACK del STM antes del timeout, se envían 2 STM y así (exp).
   c. caso contrario, VC = VC/2 (puede reducirse mucho, demora el envio x esperar timeout acks duplicados)
   d. **Retransmisión rápida:** 3 ackdup => retransmisión
2. **TCP Tahoe (internet, VC + umbral U):**
   a. timeout/3ackdups => se reduce la ventana a la mitad
   b. Comienza con arranque lento no exponencial por umbral (quizás sea mucho)
   c. Desde que alcanza el umbral el tamaño de ventana aumenta linealmente
   d. Reiniciar VC de un paq demora 1 RTT
   e. Si no hay problemas, el umbral crece hasta igualar la VC del receptor y se mantiene constante ahí
3. **TCP Reno (Evita arranque lento por timeouts):**
   a. Inicia con arranque lento
   b. VC crece linealmente hasta pérdida
   c. Retransmisión rápida
   d. **Recuperación rápida:** 1 pack-1ackdup; 1 RTT luego de que esto pasa se confirma el pack perdido.
   e. Luego de percibir el ack VC = VC/2
   f. VC ++ cada RTT.

**Duplicados retrasados:** No deben llegar estos segmentos a la CA => deben reconocerse
La situación es que un segmento S con un sequence x se demora, hay un timeout y se da el reenvío, pero SI todavía existe, luego llega y genera problemas. Entonces sea T el tiempo de vida de un paquete, Tog = segm cuyo x no se usa en Tseg. Entonces para que al volver al principio del los números de secuencia xi, no existan los segmentos viejos con xi, el espacio de secuencia debe ser grande.
Evitando problemas de interconexión por duplicados retrasados: Básicamente que el número inicial de secuencia anule la posibilidad de problemas, las posibilidades:

1. Nueva conexión => cada extremo genera un nsec de 32b aleatorio (prob de error baja)
2. Vincular el nsec al reloj del host tal que los k bits de orden mayor del reloj = nsec (como el reloj es más rápido que la asignación de nsec a los pack, el nsec de la conexión es distinto al del duplicado)

# CAPA DE RED

**Propósito**: Llevar paquetes de un host de origen a uno de destino
siguiendo una ruta conveniente
**Tareas**:
o Almacenamiento y reenvío
o Enrutamiento
o Control de congestión
o Conectar redes de distintas tecnologías
o Fragmentación

## ENFOQUES DE ENVÍO

| **Tópico** | **Enfoque** | **NO Orientado a Conexión** | **Orientado a Conexión** |
|---|---|---|---|
| **Adeptos** | | Comunidad de Internet | Empresas de telefonía |
| **Enrutamiento** | | Variable (packs siguen rutas indep.) | Fijo (definido en el CV) |
| **Config del circuito** | | No | SI (circuito virtual CV) |
| **Diseño de tablas** | | <Host dest, línea de salida> | <Host og, id CV> \| Out <host dest, id CV> |
| **Llega un pack a un router y** | | Se almacena y comprueba;<br>Se determina el router destino;<br>Se usa la entrada de la tabla del router p/envio por línea de salida; | Sigue su camino (siguiente router definido en la CV) |
| **Elementos del pack** | | Fuente y dirección de destino | Id de la CV |

| | | |
|---|---|---|
| | (info necesaria para decidir sobre la marcha) | |
| **Router colapsa y** | Los packs perdidos durante el colapso en el router se pierden | Todos los packs que pasaron por él durante la CV se pierden |
| **Ctrl de congestión** | Difícil | Sencillo si se guardaron recursos suficientes para cada CV |

**Enrutamiento jerárquico:** Forma de organizar las **tablas** de enrutamiento para que no crezcan mucho en tamaño en redes grandes, tal que se dividen los routers **por regiones**/zonas/grupos. Entonces, las tablas regionales poseen **entradas para los routers de su región, y para las otras regiones**, pudiendo así enviar packs a ellas sin conocer su estructura interna.

La **arquitectura de un router** consiste en 4 secciones:

1. Entrada: Terminacion de linea TL + Protocolo de enlace de datos PED + subsección de inspección-envío y encolado
2. Procesador de enrutamiento
3. Fabrica de conmutación de alta velocidad
4. Salida: subsección con buffer de datagramas (encolado) + PED + TL

**Conceptos básicos de alg. de enrutamiento:** Grafos con costos, y se encuentra la mejor ruta según el árbol generador de costo mínimo del algoritmo de dijkstra ejecutado en cada router, armando la tabla de enrutamiento con el resultado.

# ALGORITMOS DE ENRUTAMIENTO:

1. **Inundación:** Enviar un paquete desde el origen a través de todas sus salida, y se continúa enviando si el router al que el pack llega no es el destino. Se puede mejorar con: Registro de packs difundidos/número de secuencia/contador regresivo de saltos (para ver si se reenvía un pack recibido).

2. **LSP paquete de estado de enlace:** Se adapta a la topología y cambios en la red, así:
    a. Routers **LSR**: Cada router ejecuta dijkstra tal que: mide el costo a cada vecino y construye un pack con lo aprendido que envía a todos los otros routers y computa el camino más corto a c/u de ellos.
    b. Paquete de estado de enlace **LSP**: {senderId, sequence, age(-- x segundo),[<vecino1,retardo1>,...]} que se envía a intervalos regulares cuando cae o vuelve la línea de un vecino, distribuido con inundación (32b, verificación previa a actualización de max(sequence), la info de routers expira si esta caido) + registro de packs difundidos.
    c. **Algoritmo** de inundación LSP: Packs que llegan van a un buffer de espera, si antes de transmitirlo llega otro pack del mismo origen se comparan los

sequences y se descarta: El duplicado si son ==, el más viejo si son /=. La tabla del buffer tiene entradas <router og, sequence, data, send flags, ack flags>.

3. **Vector de distancia:** VDx(i) la distancia de x a i; R(E,x) el retardo de E a x vecinos =>**d(E,i)= min{VDxj(i) + R(E,xj)}** la min distancia de E a i a traves de algun xj=> Mv(E,i) = {xj : xj da d(E,i)}. Cada entrada de la tabla del router E es <i , d(E,i), Mv(E,i)>. Si la mejor ruta es larga y se informa un retardo corto a x desde A => se usa A; y si la métrica es # saltos las noticias se envían a 1 salto por intercambio (lineal) es decir las buenas noticias viajan rápido, pero no las malas pues no hay un límite definido para el retardo.

# CONTROL DE CONGESTIÓN:

Un paquete que llega a una línea de salida cuyo buffer está lleno, se pierde y debe retransmitirse, si esto pasa con muchos packs se pierde rendimiento. Soluciones:
**Identificar congestión:** Cada router monitorea la demora de la cola de salida tal que si supera un umbral se envía una advertencia.

1. **Regular el tráfico:** Ajustar la tasa de transferencia
    a. Packs reguladores PR, con un bit en <h> al reenviar, que pide la reducción del trafico => host al recibir deja de escuchar otro PR a ese destino x un tiempo, luego vuelve a escuchar, y si recibe más PRs sigue reduciendo el tráfico, cc lo aumenta (distancia muy larga o velocidad alta no sirve)
    b. PR salto por salto, reduce el tráfico en cada router por el que pasa hasta llegar al host og.
    c. PR con bit de advertencia en el <h> que copia el transportador en la confirmación y se reduce el tráfico. Luego de disminuir lo suficiente, el og incrementa.

2. **Desprender la carga:** De los packs menos importantes antes de que el buffer se ocupe del todo.
    a. S/App en uso se descartan los más nuevos/viejos (datatransfer/multimedia).
    b. S/importancia de pack los que tengan menor prioridad asignada.

3. **Algoritmo de detección temprana aleatoria:** Bueno cuando hosts no pueden recibir señales explícitas de congestión. Los router tiene un promedio móvil de longitudes de cola tal que si supera un umbral, se descarta al azar una fracción de packs, lo cual nota el origen x falta de acks => la CT disminuye tasa de transmisión.

# IVP4 VS IPV6

Ambos son protocolos de capa de red de internet que determinan el formato del datagrama, definen redes/direcciones IP/ tablas de reenvío, manejan la fragmentación de packs.

| Tópico | Protocolo | IPV4 | IPV6 |
|---|---|---|---|
| **sizeof(address)** | | 32b | 64b |
| **Encabezado** | | 20B + opciones | 40B |
| **Fragmentación** | | Todos los routers | Solo el host de origen |
| **<h> checksum** | | Si (en cada salto ve errores) | NO |
| **QoS** | | Limitada (campo Typeofservice) | Mejorada con etiquetas de flujo y prioridad |
| **Soporte para NAT** | | Ampliamente utilizado p/conservar direcciones | No necesario (espacio de direcciones grande) |
| **Asignación de direcciones** | | Manual/DHCP/NAT | Automática(SLAAC)/DHCPV6/ULA(privada)/link_local(entre routers) |
| **Subredes** | | Uso de máscaras (/24,/22,etc..) | Subred estándar de /64 |
| **Tabla de reenvío** | | Prefijos con CIDR más compleja y propensa al crecimiento | prefijado jerárquico (mejor TCAM) |
| **Velocidad de procesamiento** | | Menor eficiencia por campos ops y <h> checksum | Mayor x simplificación de <h> |

# OSPF VS BGP

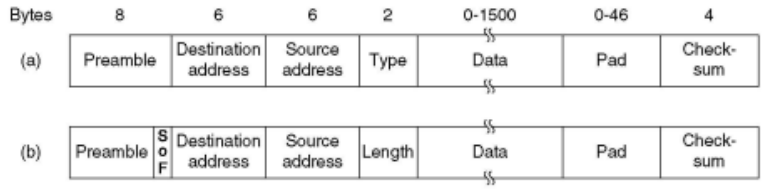
| Característica | OSPF | BGP |
|---|---|---|
| **Tipo de protocolo** | IGP (interna) | EGP (externa) |
| **Organización del área donde opera** | Dentro de un SA. jerárquicamente, con áreas dorsales | Opera entre SAs, organizada por sesiones iBGP/eBGP |
| **Modelo de enrutamiento** | Estado de enlace | Vector de rutas con atributos |
| **Decide qué ruta tomar** | Con el algoritmo de dijkstra y la info de la BDEE | En base a políticas (algoritmo de mejor ruta) |
| **Jerarquía** | Si, con áreas y red dorsal | implícitamente estructurada con ibGP/eBGP y políticas |
| **Componentes de los caminos y propagación** | Resúmenes de áreas en AEEs enviados a través de | AS_PATH; NEXT_HOP, LOCAL_PREF,MED, |

| | | |
|---|---|---|
| | los EBAs | COMMUNITY, propagados via sesiones TCP |
| **Mecanismo de actualización** | Inundación de estado de enlace con número de secuencia | Intercambio de rutas c/ sesiones TCP y mensajes UPDATE |
| **Tablas internas** | BDEE | Adj-RIB-In, Loc-RIB, Adj-RIB-Out, RIB |
| **Escalabilidad** | Limitada a redes internas organizadas en áreas | Alta (diseñado para internet global) |
| **Detección de vecinos** | Automática (msj HELLO) | Manual (config IP address y ASN del vecino) |
| **Tipos de mensajes** | Packs de datos:<br>AEE , PCEE<br>PPEE, PDBD | OPEN, UPDATE,KEEPALIVE,NOTIFICATION |
| **Manejo de cambios en la topologia** | (Rapido)Recalculando con Dijkstra y enviando info con AEEs | Segun las politicas (lento) |

# CAPA DE ENLACE

## Generalidades

| Tema | |
|---|---|
| Funciones | ● **Comm confiable** (tramas enviadas lleguen, pye/tubería),<br>● **Control de flujo** (evitar que emisor rápido sature receptor lento);<br>● **Entramado** (detección de inicio y fin de trama con banderas); Detección/Corrección de errs;<br>● **Manejo de colisiones** (transmisión de tramas simultáneamente en redes cableadas/inalámbricas) |
| Tramas | <h>(dirección origen y destino) + campo de carga util (contenido) + terminador final (control de errores) |

| | |
|---|---|
| |  <br>Formato de trama Ethernet |
| Comm entre tramas | Maquina origen: CR -bits-> CED -transmite->CED -> CR (Maq.destino)<br>La CED toma de la CR paquetes y los encapsula/divide en tramas con longitud máxima.<br>Llega trama -> el hw revisa si hay errores -> CED comprueba si es la trama esperada, y si lo es entrega el paquete dentro de la trama al software de enrutamiento, que elije la línea de salida adecuada y entrega el paquete a la CED para enviarlo.<br>Tramas de contrl para reenvio + temporizador para el reenvio (si timer pasa antes de recibir ack se reenvía, si llega primer el ack se cancela el timer)<br>Numeros de secuencia para revisar tramas duplicadas reenviadas<br>Piggybacking con trama de ack indep tras x ms. |
| Necesidad de canales de difusión | Alternativa económica para conectar maqs entre sí que quieren enviar tramas por el canal tal que todas reciben o que una envía, en lugar de conectar todas las maqs una a una.<br>Cableados (coaxial) o inalámbricos no hacer (n-1)*n conexiones teniendo n máquinas |
| Necesidad de control de colisiones | TRamas que se transmiten al mismo tiempo alteran la señal en el canal de difusión => SCAM para el manejo de colisiones; PAM para determinar acceso al canal de difusión(802.11 wifi, CSMA/CD cableadas) |

# Protocolos PAM

Protocolos de acceso múltiple que deciden quien sigue en un canal de difusión

| **Tópico** | **PAM** | **ALOHA Puro** | **CSMA persiste. 1** | **CSMA/CD** |
|---|---|---|---|---|
| **Escucha previa al canal** | | NO | Si | Si |
| **Sabe que hubo una colisiones si** | | no llega el ack => emisor espera un tiempo aleatorio y reinicia | no llega el ack => emisor espera un tiempo aleatorio y reinicia | aparecio cuando detecta a la portadora => emisor aborta y reinicia |
| **Si el canal está ocupado** | | Transmite de todos modos | Espera a que esté libre | Espera a que esté libre |
| **Rendimiento** | | Muchas colisiones | Colisiones si: no hay | Menos colisiones |

| | | | |
|---|---|---|---|
| | con carga alta | señal de uso/uso en simultáneo/tras liberación del canal | por detección inmediata, las colisiones se dan en tiempo de contención |
| **Tiempos involucrados** | Tiempo de ida y vuelta + margen pack | T de ida y vuelta +tiempo de escucha receptor | Tiempo de propagación 2t (asi definir tamaño mínimo de trama) |
| **Gestión de colisiones** | Reintentos luego de timeout | Reintentar aleatorios tras fallo | Algoritmo de retroceso exponencial binario |

# ETHERNET LAN

La capa de enlace puede hacer por hardware: Entramado, control de errores, detección de portadora/colisiones

**Algoritmo de Retroceso Exponencial Binario:** Determina el tiempo de espera tras una colisión en CSMA/CD**.**

a. Primera colisión: Estaciones esperan 0-1 tiempos de ranura (si 2 esperan lo mismo hay otra colisión)
b. Segunda colisión: Espera 0-1-2-3 tiempos
c. Tras 'i' colisiones se elige un numero aleatorio entre 0 y (exp(2,i) -1)
d. Tras 10 colisiones máximo es 1023, 16 colisiones es fracaso se notifica a la máquina y las capas superiores esperan

El retardo es pequeño cuando varias estaciones colisionan, resolución en tiempo razonable.

**Formato de trama:** 8B preámbulo + 6B dest + 6B source + 2B type + 0-1500B data + 0-46 Relleno + 4B checksum

**Ethernet conmutada:** A mayor # de estaciones mayor tráfico, por eso usamos esto.

Un conmutador es una matriz de conmutación (tabla <MAC addr, interfaz de alcance, tiempo> de hosts para aprender) de alta velocidad que almacena y reenvía tramas de ethernet sin que los hosts lo sepan. El método de filtrado y reenvío de tramas se hace según la info de la tabla tal que si ya esta una trama recibida se envía al destino, caso contrario inundación.

# REDES INALÁMBRICAS

**Tipos**

| **Característica** | **Tipo** | **AD HOC** | **DE INFRAESTRUCTURA** |
|---|---|---|---|
| **Estaciones base** | | NO | Si |

| | | |
|---|---|---|
| **Transmisión de data** | Solo dentro del alcance de un nodo | a distintas frecuencias dependiendo de la estación |
| **Organización de nodos** | Se organizan en redes (rutas entre ellos) | Cambian de Estación |

| **Característica** | **Protocolo** | **DCF** | **PCF** |
|---|---|---|---|
| **Tipo de red** | | Ad Hoc | de infraestructura |
| **Coordinador de acceso** | | Cada estación compite por el medio (distribuido) | Centralizado y controlado por la AP |
| **Técnica de acceso al medio** | | CSMA/CA | Polling coordinado por el AP |
| **Manejo de colisiones** | | RTS/CTS + retroceso exponencial binario | no hay colisiones (el AP da turno a cada estación) |
| **Problemas que aborda** | | Estación oculta y expuesta | Control total de acceso; útil en entornos congestionados |
| **Control de tráfico** | | Competencia libre por el canal (fase de contención) | El AP consulta |
| **Priorización en el acceso** | | DIFS 128us | PIFS 78us que da prioridad al AP sobre estaciones DCF |
| **Periodo de uso** | | Fase de contención | Fase sin contención |
| **Intervalos de tiempo** | | DIFS SIFS(28us) | PIFS,SIFS,CF-Poll |
| **Velocidad de respuesta/ Calidad de Servicio** | | Más lenta si hay mucha demanda<br>No garantiza mucho ancho de banda | Más eficiente en redes saturadas<br>Garantiza ancho de banda y prioridades con listas de polling |
| **Uso típico** | | Redes móviles | Redes empresariales |

| **Característica** | **CSMA/CD** | **CSMA/CA** |
|---|---|---|
| **Tipo de red** | Redes cableadas (Ethernet) | Redes inalámbricas (Wi-Fi, IEEE 802.11) |
| **Prevención o detección** | Detecta colisiones durante la transmisión | Evita colisiones antes de transmitir |

| | | |
|---|---|---|
| **Técnica principal** | La estación transmite y escucha el canal mientras envía | La estación espera y usa RTS/CTS opcionalmente antes de transmitir |
| **Colisión** | Si se detecta, se interrumpe la transmisión y se reintenta (con backoff) | Se intenta evitar que ocurra mediante control de acceso (NAV, SIFS, etc.) |
| **Necesidad de detección de portadora** | Sí | Sí |
| **Necesidad de detección de colisión** | Sí (hardware especializado) | No se puede detectar colisiones en la mayoría de los dispositivos inalámbricos |
| **Problemas que resuelve** | Evita colisiones prolongadas en medios compartidos | Resuelve problemas de estación oculta/expuesta |
| **Tamaño mínimo de trama** | Sí 64B | No necesario |
| **Rendimiento bajo carga alta** | Bueno, pero decae si muchas estaciones transmiten simultáneamente | Variable, depende del uso de RTS/CTS y calidad del medio inalámbrico |
| **Costos de colisión** | Menores, ya que se detectan y abortan rápidamente | Altos, una colisión daña el paquete completo |